

Comparative Analysis of Fractal Dimension Estimation in Diffusion-Limited Aggregation

23929

Department of Physics, University of Bath, Bath
BA2 7AY, United Kingdom
Date submitted: 25 Mar 2025

Abstract

This study evaluates methods for estimating the fractal dimension of diffusion-limited aggregation clusters of $1,000 \rightarrow 10,000$ particles, focusing on three approaches: maximum radial distance, box counting, and radius of gyration. Simulations revealed significant discrepancies in dimension, with maximum radial distance overestimating values compared to the literature (1.738 ± 0.003 for 1000 particles; 1.745 ± 0.001 for 5000 particles) due to lattice anisotropy and emergent directional growth bias, while box counting underestimated (1.567 ± 0.005) owing to insufficient resolution and averaging artifacts. Implementing the radius of gyration aligned closest to the literature value of ~ 1.71 (1.710 ± 0.000), as it mitigated anisotropy by averaging spatial distributions. Sticking probability further influenced results, with low probability amplifying lattice effects and high probability promoting screening. The findings underscore the sensitivity of fractal dimension to measurement protocols and lattice constraints, advocating for off-lattice simulations and larger clusters for analysis.

Introduction

Naturally occurring structures are often constrained in their growth via diffusive processes. In 1981, the model of diffusion-limited aggregation (DLA) was introduced by Witten and Sander [1], facilitating the computational study of related phenomena such as electrodeposition, bacterial growth, colloidal aggregation, etc [2].

The model is defined as follows: A seed particle is generated at the origin of a regular two-dimensional lattice. Another particle, sufficiently far away, begins a random walk (i.e., diffuses) until adjacent to an occupied site. Here it becomes stationary; bound to its neighbour. Another particle is then launched, thereby repeating the procedure until an indefinitely large cluster may be formed.

This model functions, but generating a cluster can take an exhaustively long time. Here, to reduce computation time, three concentric circles centred at the origin are established: the “cluster circle”, the “starting circle”, and the “killing circle”. The cluster circle defines the current approximate size of the aggregate and expands as the cluster grows. New particles are not produced

randomly across the entire lattice but instead are released from random positions along the starting circle, which lies slightly beyond the cluster circle. This ensures that particles begin close enough to have a reasonable chance of joining the cluster. This has no effect on the structure being no different probabilistically than a particle arriving randomly from elsewhere, the distribution over 360° being the same.

The killing circle lies further out. If a particle diffuses beyond this outermost boundary without sticking to the cluster, it is removed from the simulation and replaced with a new particle launched from the starting circle. This prevents computational effort from being wasted on particles that have effectively wandered too far to ever return. The inclusion of the killing circle can alter cluster properties, but these changes can be mitigated by ensuring its remoteness to the cluster circle [3]. Figure 1 illustrates this procedure, taking the form of a simple square lattice version of the DLA model.

The clusters formed through DLA exhibit complex, multi-branched structures that emerge from the stochastic nature of the growth process. A key property

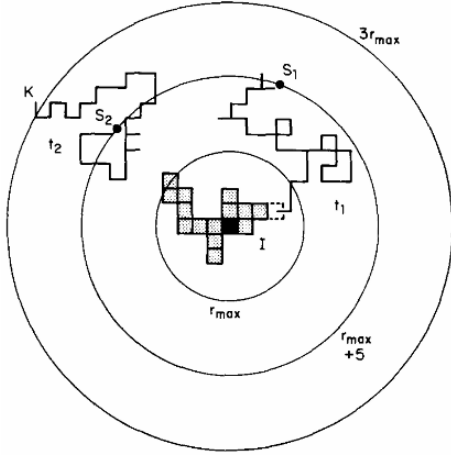


Figure 1. A schematic of a square lattice DLA model. The initial seed site is shown in black, whilst subsequent sites are shaded. Two example trajectories, t_1 and t_2 , are depicted, originating from points S_1 and S_2 . Trajectory t_1 leads to the cluster whereas t_2 strays outwards and is terminated upon contact with the outer bound. Here, $r_{\max}$, $r_{\max} + 5$ and $3r_{\max}$ signify the radii of the cluster circle, starting circle and killing circle, respectively [4].

of these aggregates is scale invariance: the statistical properties of the cluster remain unchanged under a rescaling of length, meaning that portions of the structure resemble the whole at different magnifications [1, 5].

A natural consequence of the scale-invariant geometry of DLA clusters is that their growth follows a power-law relation characteristic of fractal structures. In classical Euclidean objects, the number of unit-size particles N_c required to build a structure scales as a power of the system size R : for a line, $N_c \propto R$; for a square, $N_c \propto R^2$; and for a cube, $N_c \propto R^3$. This scaling can be generalised to fractal objects, where the number of particles scales as:

$$N_c \propto \left(\frac{R}{a}\right)^{d_f} \quad (1)$$

Here, a is the particle size, and d_f is the fractal dimension of the object, which can be non-integer. Rearranging gives:

$$d_f = \frac{\ln N_c}{\ln(R/a)} \quad (2)$$

In simulations of DLA, a common approach is to estimate R using the maximum distance of any particle from the origin, $r_{\max}$, as shown in figure 1. However, due to statistical noise or finite-size effects that dominate small clusters, the simple power-law relation may not hold precisely. To accommodate this, a more flexible model is used:

$$N_c(r_{\max}) = (\alpha r_{\max})^{d_f} + \beta \quad (3)$$

Where α and β are unknown constants. Taking logarithms and differentiating with respect to $\ln r_{\max}$:

$$\frac{d(\ln N_c)}{d(\ln r_{\max})} = \frac{d_f}{1 + \beta/(\alpha r_{\max})^{d_f}} \quad (4)$$

This expression shows that the derivative approaches d_f as $r_{\max}$ increases. Therefore, the fractal dimension can be estimated by plotting $\ln N_c$ against $\ln r_{\max}$ and measuring the slope as $r_{\max}$ gets large. If $\beta = 0$, the slope directly gives d_f , regardless of the value of α [3].

Determining fractal dimension

The DLA simulation was initialised at 1000 particle clusters for 1000 runs. The C++ code (provided by A. Nogaret, D. Tsang and A. Walker) utilised a seed to generate random clusters for each iteration; to avoid the same pattern being formed by the “random” number generator the seed was programmed to change dynamically. As each cluster grew, values of N_c and $r_{\max}$ were logged at small intervals of N_c . After computing the natural logarithms, the first four initial runs were plotted. Figure 2 shows the approximately linear relationship observed across runs, confirming the expected scaling behaviour and self-similar nature of the growth process.

To reduce statistical fluctuations and obtain a precise estimate for d_f , the data was aggregated by binning values for the same $\ln(N_c)$ and computing the average $\ln(r_{\max})$ along with standard errors. The resulting dataset was used to perform an orthogonal distance regression (ODR), accounting for the uncertainties in the x-axis. Figure 3 displays the dataset alongside the best-fit line obtained from ODR. From this fit, the slope, interpreted as the fractal dimension d_f , was found to be 1.738 ± 0.003 .

The number of particles was then increased to 5000 and a similar plot was obtained, as seen in figure 4. A value of $d_f = 1.745 \pm 0.001$ was determined from ODR.

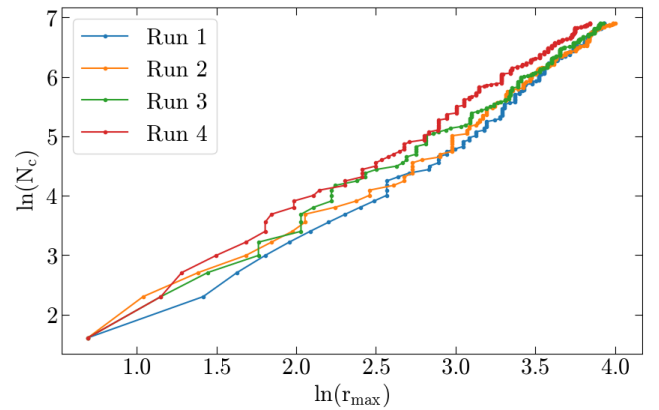


Figure 2. Log-log plot of the number of particles against the cluster radius. The first four of 1000 runs are shown, demonstrating the stochasticity of independent DLA aggregates.

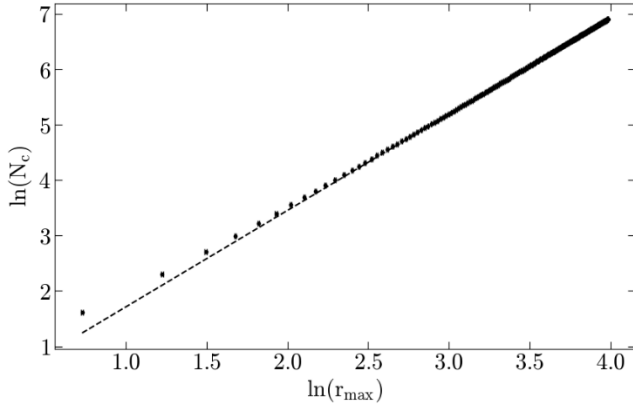


Figure 3. Log-log plot of the number of particles against the average logarithm of the cluster radius. 1000 particles were averaged over 1000 runs. Error bars are plotted but not distinguishable. At small length scales the breakdown of the model can be seen, with data points straying from the regression line. $d_f = 1.738 \pm 0.003$.

Both dimensions obtained were above the literature value of ~ 1.71 [6], a fault with the method that is discussed later. Furthermore, there was an increase in dimension when going from 1000 to 5000 particles. Successive linear portions of figure 4 were analysed and remained constant towards the latter half of data, indicating that the cluster reached saturation at low particle numbers, $\sim 10^3$.

Investigating sticking Probability

Sticking probability in DLA systems represents a variety of real-world processes where particles, molecules, or entities do not always attach to a growing structure or surface. For example, in electrodeposition, the sticking probability represents the likelihood that an ion will adhere to the electrode surface upon contact. Controlling the sticking probability can ensure uniform and high-quality coatings in manufacturing procedures.

Its implementation is simple: When a particle encounters an occupied site the four neighbouring sites are checked for occupancy. For each neighbour, a random number between 1 and 100,000 is generated. If the random number divided by 100,000 is less than the sticking probability, p , the particle sticks to the cluster. However, if the number is greater, it will not stick, and the particle will continue its random walk.

The sticking probability plays a pivotal role in determining the shape and structure of the resulting cluster. When $p \rightarrow 1$, particles tend to stick immediately upon encountering the cluster's surface. This leads to rapid growth at the outermost tips of branches, creating long, thin arms that extend outward. However, these protruding branches also function as shields, preventing new particles from reaching the

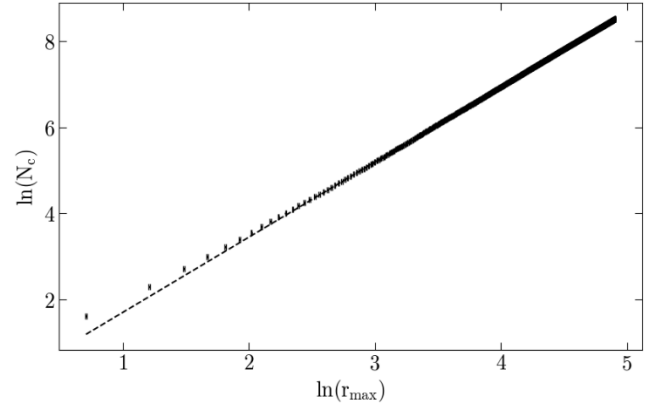


Figure 4. Log-log plot of the number of particles against the average logarithm of the cluster radius. Here, the simulation ran for up to 5000 particles which were again averaged over 1000 runs. Error bars are plotted but negligible. $d_f = 1.745 \pm 0.001$.

interior regions of the cluster. This phenomenon is termed “screening” [7].

Conversely, when $p \rightarrow 0$, particles undergo prolonged random walks before attaching to the cluster. This allows them to navigate around branch tips and explore deeper into the cluster's interior, including the narrow, winding channels known as “fjords” [6]. As particles meander into these fjords, they stick to their walls or ends, effectively filling in the gaps and smoothing the cluster's edges. This reduces the overall fractal complexity, resulting in a denser, bushier cluster with fewer gaps and shorter branches as illustrated in figure 6. Such clusters have a higher fractal dimension.

For each sticking probability, the C++ simulation was ran 1000 times for a 1000 particles. After averaging, log-log plots were produced, and the fractal dimensions and their errors were determined from the slopes using the same ODR as before. However, the values of r_{max}

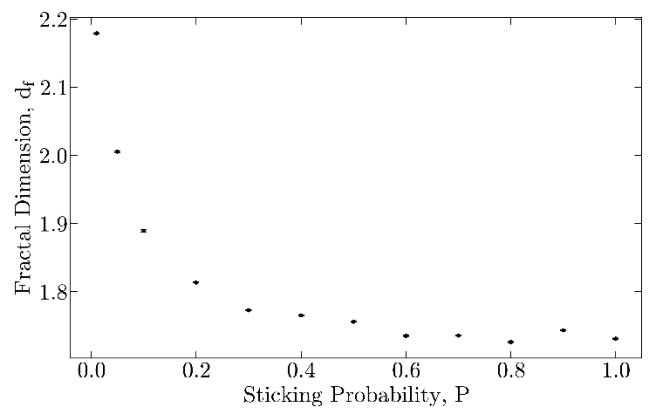


Figure 5. Fractal dimension, determined via log-log plot as previously, plotted against the sticking probability. Despite being embedded in Euclidean 2D geometry, the fractal dimension rises above 2, demonstrating issues with the current method.

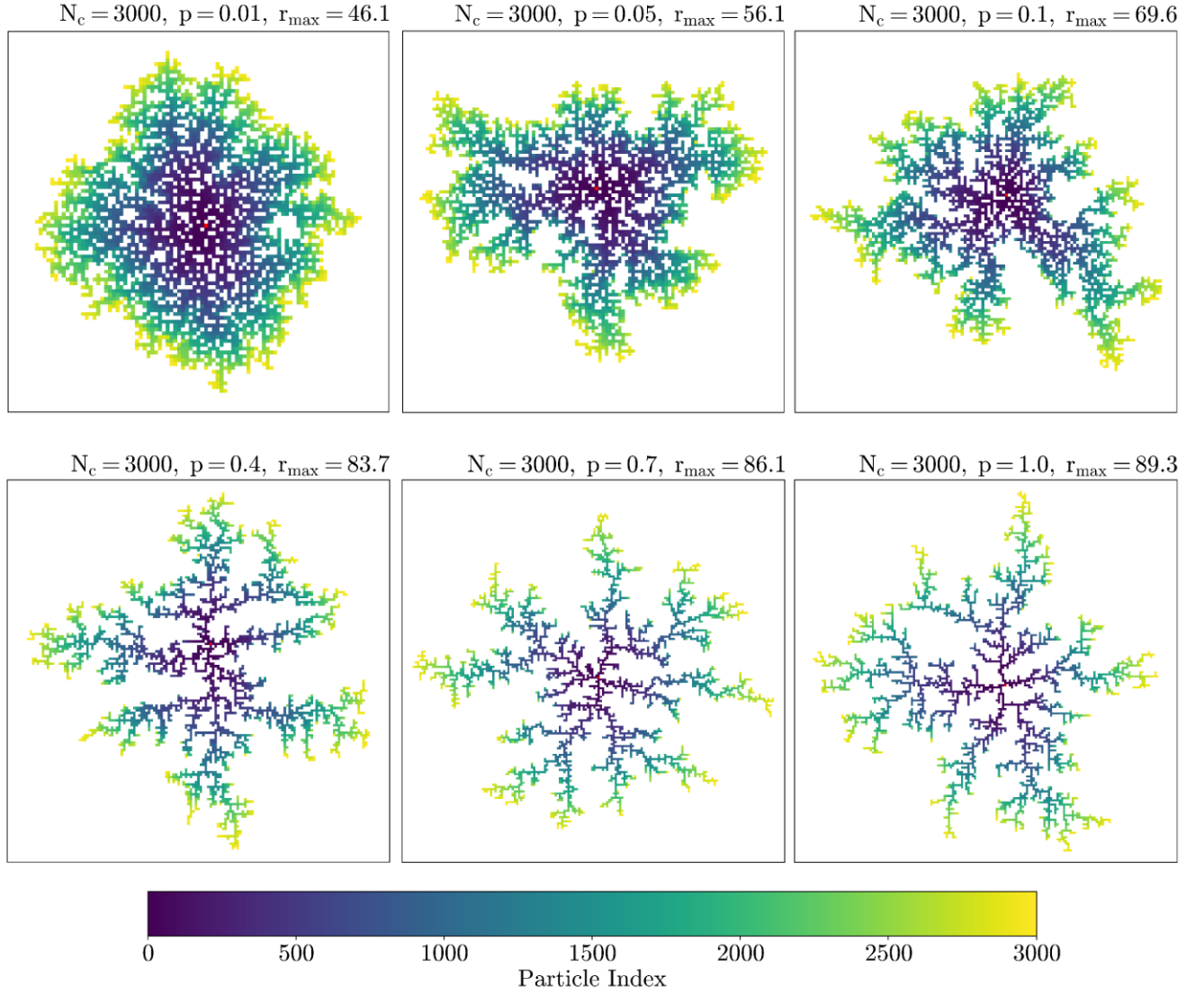


Figure 6. Depiction of typical 3000 particle clusters for increasing sticking probability, p . The red patch at the centre of clusters represents the seed particle. As p decreases the cluster becomes more susceptible to lattice anisotropy, causing more growth along the x and y axes. Colour gradient represents order of particle addition to the cluster and demonstrates the screening effecting that concentrates new growth at the tips.

for $N_c < 300$ were excluded to eliminate the non-linear data from influencing d_f . These results were then plotted in figure 5.

needed to the object scales with the box size ϵ according to the relationship:

$$N_c(\epsilon) \sim \epsilon^{-d_f} \quad (5)$$

Due to the nature of a square lattice, the fractal dimension is artificially enlarged because of the effects of lattice anisotropy. As can be seen in figure 6, although at high p the cluster more closely resembles an off-lattice structure, at low p the value of r_{max} becomes even more inflated due to growth bias in the x and y directions. This heightened anisotropy at low p is what leads to these unphysical results for fractal dimension.

Box Counting

Box counting is a computational model that measures how the number of boxes required to cover a fractal object changes as the size of the boxes decreases. The key idea is that for a fractal, the number of boxes $N(\epsilon)$

The process begins by representing the cluster as a set of points in 2D space, such as (x, y) coordinates. For each box size ϵ , ranging from 1 to a specified maximum, the space is divided into a grid of boxes of that size. Each point is then assigned to a box by dividing its coordinates by ϵ and taking the floor of the result, ensuring that the box coordinates are unique. The number of unique boxes $N_c(\epsilon)$ required to cover all points is counted and recorded for each box size. This process is repeated for increasing box sizes. By plotting $\ln(N_c(\epsilon))$ against $\ln(\epsilon)$, d_f can be extracted from the slope [8].

Table I: Variation in fractal dimension with decreasing sticking probability using box counting method.

Sticking Probability, P	Fractal Dimension, d_f	Error ($\pm$)
0.01	1.802	0.010
0.05	1.710	0.006
0.10	1.660	0.004
0.50	1.599	0.003
1.00	1.567	0.005

Although the values recorded in Table I do not exceed 2, they also vastly underestimate the true values for the fractal dimension.

In this case, 10,000 particles were produced for 10 runs at each p value. Box counting becomes ineffective for small clusters due to limitations in both scale and resolution. This sample size is simply too limited to span a wide range of box sizes, which is crucial for establishing a reliable scaling relationship in the log-log plot used to estimate fractal dimension. The boxes themselves also fail to meaningfully cover multiple width scales, resulting in too few data points for an accurate slope calculation. Additionally, because small clusters haven't developed much structural complexity, box counting tends to miss important fine-scale features. Combined with the discrete nature of the grid and particle placement, this leads to distorted counts and ultimately unreliable estimates of the fractal dimension.

Radius of gyration

The radius of gyration, R_g , is a fundamental metric in the study of fractal growth processes like DLA. It quantifies the spatial distribution of particles in a cluster by measuring their average distance from the cluster's center of mass, weighted by their positions. Mathematically, it is defined as:

$$R_g = \sqrt{\frac{1}{N_c} \sum_{i=1}^{N_c} |\mathbf{r}_i - \mathbf{r}_{\text{com}}|^2} \quad (6)$$

Where $\mathbf{r}_i$ is the i^{th} particle position, and $\mathbf{r}_{\text{com}}$ is the centre of mass of the cluster for each value of N_c [7].

Table II: Variation in fractal dimension with decreasing sticking probability using radius of gyration rather than maximum radial distance.

Sticking Probability, P	Fractal Dimension, d_f	Error ($\pm$)
0.01	2.001	0.001
0.05	1.855	0.001
0.10	1.781	0.000
0.50	1.717	0.000
1.00	1.710	0.000

Replacing r_{max} with R_g , for the same dataset, mitigated the effects of lattice anisotropy due to being less sensitive to directional spikes. This resulted in values much closer to the published value, especially at $p = 1$, which concurs with $d_f = 1.71$ from the literature. However, while it does reduce these growth biases at low p , it does not make them negligible and therefore the fractal dimension is still slightly inflated and crosses the upper bound of 2.

Discussion and conclusion

The results from the three methods: maximum radial distance, box counting, and radius of gyration, reveal significant discrepancies in the estimated fractal dimension, highlighting the influence of measurement techniques and lattice effects on DLA analysis. The r_{max} method produced values slightly above the literature benchmark ($d_f \sim 1.71$), yielding 1.738 ± 0.003 for 1000 particles and 1.745 ± 0.001 for 5000 particles, likely due to lattice anisotropy inflating radial distances, particularly at low sticking probabilities. Box counting, while theoretically robust, underestimated d_f because of insufficient resolution for small clusters and spatial averaging that obscured fine fractal features. In contrast, the R_g method aligned best with expected values (1.710 ± 0.000) as it averages particle distributions, mitigating directional bias. However, its deviation at low p ($d_f > 2$) suggests lingering lattice artifacts. The unusually small regression errors, due to pre-averaging data, imply overconfidence in precision; a more rigorous approach would compute d_f per cluster before statistical aggregation to capture true variability. Future work should prioritise off-lattice simulations, larger clusters, and alternative metrics like sandbox methods to reduce systematic biases [7]. Ultimately, while R_g offers the most reliable estimates, all methods require careful scrutiny of their assumptions and limitations in fractal analysis.

References:

- [1] Witten T A and Sander L M 1981 Diffusion-limited aggregation, a kinetic critical phenomenon *Phys. Rev. Lett.* **47** 1400–3
- [2] Ball R C and Blumenfeld R 1991 Exact results on exponential screening in two-dimensional diffusion-limited aggregation *Phys. Rev. A* **44** R828–31
- [3] PH30056 Comp Phys B– DLA Coursework Assignment, University of Bath
- [4] Meakin P 1995 Progress in DLA research *Physica D* **86** 104–12
- [5] Witten T A and Sander L M 1983 Diffusion-limited aggregation *Phys. Rev. B* **27** 5686–97
- [6] Halsey C H 2000 Diffusion-limited aggregation: a model for pattern formation *Phys. Today* **53** (11) 36–41
- [7] Vicsek T 1992 *Fractal growth phenomena* 2nd ed. (Singapore: World Scientific)
- [8] Hibbert D B 1991 Fractals in chemistry *Chemometrics and Intelligent Laboratory Systems*. **11** 1–11.

Appendix / Appendices

Sticking Probability Pseudocode:

```
Function checkStick(particle):  
    p = 0.05 // Sticking probability  
    for each neighbor of particle:  
        if neighbor is occupied:  
            x = random number between 1 and 100,000  
            if x / 100,000 ≤ p:  
                return True // Particle sticks  
    return False // Particle does not stick
```

Box Counting Pseudocode:

```
function box_counting(points, max_box_size):  
    # Initialize an empty list to store results  
    results = []  
  
    # Iterate over box sizes from 1 to max_box_size  
    for box_size in range(1, max_box_size + 1):  
        # Create a set to store unique boxes  
        boxes = set()  
  
        # Iterate over all points  
        for (x, y) in points:  
            # Calculate which box the point belongs to  
            box_x = floor(x / box_size)  
            box_y = floor(y / box_size)  
  
            # Add the box to the set  
            boxes.add((box_x, box_y))  
  
        # Count the number of unique boxes  
        num_boxes = length(boxes)  
  
        # Store the result for this box size  
        results.append((box_size, num_boxes))  
  
    # Return the list of results  
    return results
```